

BloodLink Security Audit

Plain-language report · updated 19 September 2026 · static review + authorized dynamic test on a local instance

Where things stand. The urgent exposure and every “a stranger can read the medical data or take the site down” problem have been fixed and pushed. The live donor database has been removed from the public code history; the login, sign-up, password-reset and deployment weaknesses are closed; and the two issues found during live testing (the old donation route and an id-overflow error) are now fixed too. What remains is either an action only the repository owner can take (make the repo private, ask GitHub to purge caches, rotate the keys) or lower-priority engineering work that does not expose data. This report explains each item in plain terms and marks its current status.

How to read the tags

CRITICAL	Real harm already possible.	FIXED	Done and pushed.
HIGH	Outsider could break or corrupt.	PARTIAL	Code done; a step remains.
MEDIUM	Needs conditions or an insider.	USER	Only the owner can do it.
LOW	Small gap / rough edge.	OPEN	Not started yet.

What was tested this round

The whole codebase was reviewed, and an authorized dynamic test was run against a throwaway local copy of the app (real HTTP attacks: authorization, IDOR, injection, the enumeration and lockout defenses, the donation route, headers, rate limits). The automated test suite now has 133 passing tests. Note: a live test on a throwaway copy is still not a substitute for an independent third-party penetration test before real medical data is accepted.

Fixed and pushed

CRITICAL

FIXED

C1. Donor database was in the public code history

What it is

The app's real database (172 donors with names, phone numbers, date of birth, address and medical screening, plus the admin's scrambled password and 41 email links) had been committed to the project history and was downloadable by anyone.

Why it matters

Personal, health-related data about real students, readable by anyone who cloned the public repository.

Status & action

The entire history was rewritten to remove the file: all 67 commits preserved, zero database objects remain, verified from a fresh clone; the old commits are unreachable and the local backup was securely deleted. **Still on the owner:** ask GitHub Support to purge cached copies, rotate the app secret and admin password, and set the repository to Private.

For the developer: `git filter-repo mirror rewrite; force-pushed; new head 6515cd1; git rev-list --all --objects | grep bloodlink.db` is empty.

HIGH

FIXED

H1. One person could lock everyone out (on Render)

What it is

Because the app was not told each visitor's real address behind Render's proxy, all visitors shared one throttle counter; 30 failed attempts would block sign-in for everybody.

Why it matters

During an emergency blood request nobody could sign in.

Status & action

The app now trusts Render's private network so each visitor is counted separately, and an admin-only diagnostics page lets you verify this after deploying. `render.yaml` and the deployment guide were updated.

For the developer: `start.py forwarded_allow_ips default; GET /api/admin/diagnostics/request; verified live.`

HIGH

FIXED

H2. Anyone could lock the administrator out

What it is

Failed logins were counted per username, so a stranger could keep the admin permanently locked out by guessing the known default username.

Why it matters

Denial of the admin account, indefinitely.

Status & action

Failures are now counted per (visitor + account) pair and cleared on a successful login, so a stranger can only affect pairs they themselves use. The 429 response is identical for real and fake usernames, so it reveals nothing.

For the developer: `backend/routers/auth.py login-failures bucket; login-account-watch tripwire.`

HIGH**FIXED****H3. Old donation route skipped safety checks****What it is**

A legacy admin route accepted negative or absurd unit counts, future dates, and did not check blood-group compatibility; it also left the request re-recordable.

Why it matters

Corrupted statistics and a bypass of the blood-compatibility (patient-safety) check.

Status & action

The route now bounds units (1-10), restricts the donation type, rejects future dates, enforces donor-to-request blood compatibility, and refuses an already-completed request; oversized numbers no longer error. Found in live testing on 18 Sep, fixed and pushed 19 Sep.

For the developer: `backend/routers/donations.py + DonationRecordCreate schema; status Completed -> Fulfilled; 133 tests.`

MEDIUM**FIXED****M4. Strangers could learn who is a registered donor****What it is**

The sign-up form gave different errors for a taken username vs. email vs. phone, and the reset form responded faster for real accounts, so someone could confirm whether a person is on the donor list.

Why it matters

Sensitive: reveals membership of a blood-donor (health) list.

Status & action

Sign-up now returns one generic conflict message, and reset/setup emails are sent in the background so timing is identical whether or not the account exists.

For the developer: `REGISTRATION_CONFLICT_MESSAGE; BackgroundTasks in password_reset.py and donor_registration.py.`

MEDIUM**FIXED****M5. Reset / setup links could be abused to flood or sabotage****What it is**

There was no per-account limit on reset emails (inbox flooding with real links), and anyone knowing a pending username could keep cancelling the victim's setup link.

Why it matters

Harassment, and blocking a new donor from completing setup.

Status & action

Reset and setup emails are now capped at three per account per hour, and a still-valid setup link is never replaced.

For the developer: `consume_limit password-reset-account / setup-email-account; issue_password_setup_token returns None while a live token exists.`

MEDIUM**FIXED****M3. Dashboard would crash on the production database****What it is**

The dashboard used a date function that only exists in the local (SQLite) database, not the production one (PostgreSQL).

Why it matters

The admin dashboard would show an error once live.

Status & action

Replaced with a portable month-range query that works on both databases and uses the date index.

For the developer: `backend/database/crud.py get_donation_dashboard_summary`; Python-computed month bounds.

MEDIUM**FIXED****A4. Admin password hardening and self-service change****What it is**

The admin used a guessable default username, the bootstrap password could linger in the environment, and there was no way for a user to change their own password.

Why it matters

A weak or leftover admin password is the main way in, since there is no MFA yet.

Status & action

Production now refuses guessable admin usernames and passwords under 16 characters, warns if the bootstrap password is left set, and users can change their own password (which signs out every session).

For the developer: `settings.get_default_volunteer_credentials`; POST `/api/auth/change-password`; Settings -> Security UI.

LOW**FIXED****LOW-1. Oversized id numbers returned a server error****What it is**

A very large number in a URL (e.g. `/api/donors/9999...`) caused an internal error instead of a clean “not found”.

Why it matters

Cosmetic; admin-only; no data exposure.

Status & action

Such ids now return a clean 404 on every endpoint; genuine database errors return a generic message with no stack trace. Found in live testing on 18 Sep, fixed 19 Sep.

For the developer: `backend/main.py OverflowError / DataError / OperationalError handlers` -> 404.

Actions only the repository owner can take

These are not code changes; they need access to GitHub, Render, and the data-owner relationship.

- **Set the GitHub repository to Private.** It was still public at the time of writing. This immediately stops new visitors and search engines, and also contains this very report (a security document is a roadmap for attackers).
- **Ask GitHub Support to purge cached views** for the repository after the history rewrite. Until they do, the old commits can still be opened by direct link even though no clone fetches them.
- **Rotate the secrets that were exposed:** regenerate the app SECRET_KEY on Render, and change the administrator password (via Settings → Security) after deploying.
- **Set the new deployment variables** from render.yaml (FORWARDED_ALLOW_IPS, ALLOWED_HOSTS, a non-default admin username) and remove DEFAULT_VOLUNTEER_PASSWORD once the admin account exists.
- **Ask anyone with an old clone to re-clone** — old copies still contain the donor data.
- **Turn on two-factor authentication** on the Render and GitHub accounts themselves.

Remaining engineering work (no data exposure)

Real improvements, but none of these let an outsider reach the medical data. Roughly in priority order.

MEDIUM

OPEN

MFA. Admin two-factor authentication

What it is

The admin password is currently the only lock on the door; there is no second factor in the app.

Why it matters

A phished or reused admin password would expose all donor data.

Status & action

Planned as its own piece of work (~4 hours: enrolment, second login step, recovery codes). Meanwhile, enabling 2FA on the Render and GitHub accounts covers the infrastructure path today.

MEDIUM

OPEN

M1/M2. Donation points race and eligibility

What it is

Two simultaneous confirmations can lose points, and the 90-day wait between donations is not enforced at confirmation.

Why it matters

Untrustworthy points/leaderboard; a donor could be recorded again too soon.

Status & action

Make the points update atomic and enforce the interval and donor status at confirmation time.

MEDIUM

OPEN

M6. It is possible to end up with no administrator

What it is

An admin can demote or delete the last admin, and the bootstrap only runs on an empty database.

Why it matters

Recovery would need manual database surgery. Accounts also cannot be disabled, only deleted.

Status & action

Add a “last administrator” guard and an enable/disable switch for accounts.

- **Pagination and fan-out (M8, LOW-MED).** List endpoints return everything, and campaigns send emails one by one in the request. Add page limits and background sending.
- **Dependency pinning (M11, MED).** Pin every library with hashes; replace the unmaintained password library.
- **Continuous integration (M13, MED).** Nothing runs the tests before deploy yet — add a GitHub Actions workflow so the 133 tests gate every change.
- **Deletion is not erasure (M10, MED).** Decide what “delete a donor” should mean and make the code match.
- **Smaller items (LOW):** spreadsheet-import row cap can be under-declared; loose enum/date validation on some fields; look-alike usernames for admins; the SECURITY.md claim about a hardened XML parser is still inaccurate.

What is confirmed working well

Confirmed both by code review and by live attack against a running copy:

- Database queries cannot be injected; login tokens are signed, expire, and are purpose-scoped.
- Sessions live in a secure browser-only cookie; logging out or any password/role change kills every session instantly.
- Authorization holds: donors and anonymous users are correctly blocked from all admin data and actions; no IDOR found.
- Patient identity and contact are hidden from donors; passwords are bcrypt-hashed; one-time links really are one-time.
- Push notifications can only reach known providers (no server-side request forgery); emails escape all content.
- The web pages escape all data, load no third-party scripts, and never cache personal data offline.
- The blood-type compatibility table is correct for all eight groups, and is now enforced on the donation route too.

Bottom line

No grave or exposure-class risk remains in the code. Every Tier 1 and Tier 2 item is fixed and pushed, and the two issues found during live testing are fixed. The most important things left are the owner actions above — especially making the repository private and rotating the exposed secrets — and, next, admin two-factor authentication. A final independent penetration test with the real production configuration is still recommended before accepting real donor data.